

LabTrack

Roteiro passo a passo para aprender arquitetura construindo um sistema de reservas e empréstimos de laboratório.

Dominio simples. Decisoes tecnicas reais. Cada etapa vira codigo, evidencia e uma conversa de arquitetura.

Cadencia	35 semanas produtivas, 4-5 h/semana. Inclui duas janelas de buffer.
Regra do jogo	Toda tecnologia do roadmap precisa resolver um problema observavel ou existir como laboratorio isolado.
Stack-base	TypeScript, Node.js, Next.js, PostgreSQL, Redis, RabbitMQ, Docker e Terraform.
Resultado	Um produto demonstravel e um portfolio de ADRs, diagramas, testes e operacao.

Como usar este roteiro

Este não é um checklist para instalar tecnologias. É um roteiro de engenharia: você lê o mínimo necessário, toma uma decisão explícita, constrói uma parte pequena, mede o efeito e registra o que aprendeu. O LabTrack mantém as regras de negócio propositalmente curtas para deixar a arquitetura ser o desafio.

Ciclo de cada capítulo

1. Ler e tomar notas. 2. Desenhar C4, UML ou fluxo. 3. Registrar uma ADR. 4. Implementar o menor corte vertical. 5. Testar e demonstrar. 6. Defender a decisão em cinco minutos.

Definição de pronto

Artefato	Pergunta que ele responde	Mínimo aceitável
ADR	Por que esta decisão?	Contexto, opções, decisão, consequências e como reverter.
Diagrama	Quem conversa com quem?	C4 de contexto/container ou sequência para o fluxo alterado.
Teste	Como sei que a regra continua válida?	Teste automatizado da regra ou do contrato relevante.
Demo	O que mudou para o usuário ou operador?	Cenário feliz e um cenário de falha/concorrência.
Métrica	O que melhorou ou piorou?	Ao menos uma medida: p95, taxa de erro, fila, custo ou tempo de recuperação.

O que fica no produto e o que vira laboratório

REST, PostgreSQL, controle de empréstimo, trilha de auditoria, notificações e operação entram no produto.

Tópicos que seriam artificiais no domínio - por exemplo SOAP, Hadoop/MapReduce ou microfrontends - entram em laboratórios pequenos, com dados simulados e fronteira clara. Você aprende o conceito sem deformar o LabTrack.

A empresa e o problema

A Orion Labs opera tres laboratorios compartilhados. Cerca de 350 itens - notebooks, cameras, projetores, kits de robotica e ferramentas - circulam entre pesquisadores e instrutores. Em dias cheios chegam a 80 pedidos. Hoje, planilhas e mensagens resolvem o basico, mas produzem conflitos: duas pessoas aparecem para o mesmo equipamento, um item danificado continua como disponivel e ninguem sabe com certeza quem esta com ele.

A diretora de operacoes aprova o LabTrack como um produto interno. A primeira versao nao tenta resolver compras, faturamento ou contratos. Ela precisa mostrar disponibilidade confiavel, permitir solicitacao, aprovar, retirar, devolver e tratar manutencao. A cada capitulo, o crescimento de uso cria uma justificativa concreta para uma decisao arquitetural.

Solicitante	Consulta itens, pede reserva e acompanha seu emprestimo.
Gestor do laboratorio	Aprova solicitacoes e confirma retirada/devolucao.
Tecnico de manutencao	Coloca item em manutencao e registra sua liberacao.
Administrador	Mantem catalogo, perfis e parametros operacionais.

Regras de negocio: manter simples

ID	Regra	Como validar
RB-01	Um ativo fisico nao pode ter dois emprestimos ativos ao mesmo tempo.	Teste de concorrencia e restricao no banco.
RB-02	Ativo em manutencao nao pode ser reservado nem emprestado.	Teste de transicao de estado.
RB-03	Somente gestor aprova solicitacoes e confirma retirada/devolucao.	Teste de autorizacao por papel.
RB-04	Retirada, devolucao e manutencao geram historico imutavel.	Teste de auditoria e consulta de trilha.
RB-05	Nao devolvido ate a data prevista fica atrasado.	Job agendado e teste de tempo.

Estados minimos

Entidade	Estados
Ativo	AVAILABLE, RESERVED, LOANED, IN_MAINTENANCE, RETIRED
Emprestimo	REQUESTED, APPROVED, READY_FOR_PICKUP, CHECKED_OUT, RETURNED, OVERDUE, REJECTED, CANCELLED

Bussola arquitetural do LabTrack

Comece como um monolito modular. A escolha é deliberada: o domínio é pequeno, a equipe é pequena e você precisa aprender a estabelecer fronteiras antes de distribuí-las. Os capítulos posteriores testam limites com filas, cache, serviços e protocolos, mas não obrigam a produção a virar um sistema distribuído sem necessidade.

Camada	Responsabilidade	Tecnologia inicial
Experiência	Portal do solicitante e painel do gestor.	Next.js + TypeScript
Aplicação	Casos de uso, autorização, contratos e orquestração.	Node.js + Fastify/Nest/Express
Domínio	Ativo, empréstimo, transições e políticas.	TypeScript, testes unitários
Infraestrutura	PostgreSQL, Redis, fila, e-mail simulado, observabilidade.	Docker Compose
Plataforma	Provisionamento, pipeline, deploy e recuperação.	Terraform + CI/CD

Princípio central

Acoplamento baixo não significa criar muitos serviços. Significa que uma mudança em notificação, banco ou interface não altera a regra de que um item não pode estar emprestado duas vezes.

Mapa dos capítulos

Bloco	Capítulos	Pergunta principal
Descoberta e núcleo	01-03	Que produto vamos construir e onde moram as regras?
Correção e segurança	04-05	Como evitar conflito e acesso indevido?
Integração e assincronia	06-07	Como expor, integrar e processar sem travar o fluxo?
Escala e distribuição	08-10	Como reagir a uso maior sem complexidade gratuita?
Experiência e operação	11-12	Como entregar uma interface e uma plataforma operável?
Arquitetura corporativa	13	Como conectar o produto ao restante da organização?

Cronograma e deadlines

Planejamento de referencia: 35 semanas produtivas, a partir de 28/09/2026, com 4-5 horas por semana. Os dois buffers existem para corrigir defeitos ou aprofundar uma decisao; nao sao convites para acumular escopo.

Cap.	Janela	Deadline	Marco
01	28/09 - 11/10	11/10	Contexto, atores, RF/RNF e riscos.
02	12/10 - 01/11	01/11	MVP navegavel de reserva e gestao.
03	02/11 - 22/11	22/11	Dominio testado e arquitetura em camadas.
04	23/11 - 13/12	13/12	Consistencia, transacoes e carga concorrente.
Buffer	14/12 - 03/01	03/01	Recuperar atraso ou consolidar documentacao.
05	04/01 - 17/01	17/01	Autenticacao, autorizacao e auditoria.
06	18/01 - 07/02	07/02	APIs, contratos e protocolos comparados.
07	08/02 - 28/02	28/02	Fila, worker e notificacao confiavel.
08	01/03 - 21/03	21/03	Cache, serverless e tarefas agendadas.
09	22/03 - 11/04	11/04	Dados, analise e ETL de eventos.
10	12/04 - 02/05	02/05	Distribuicao, CQRS e limites de consistencia.
11	03/05 - 16/05	16/05	Estrategias web e qualidade da interface.
12	17/05 - 06/06	06/06	Cloud, IaC, CI/CD, rede e operacao.
13	07/06 - 20/06	20/06	Visao corporativa e defesa final.
Buffer final	21/06 - 04/07	04/07	Hardening, portfolio e retrospectiva.

Regra de prazo

Se um capitulo nao couber no prazo, corte a largura da funcionalidade, nao a evidencia. Um fluxo menor com ADR, teste e demo vale mais que uma tela grande sem decisao verificavel.

Descoberta, stakeholders e requisitos

Por que agora?

Antes de escrever endpoints, o gestor precisa provar que o produto ataca as falhas reais do laboratorio e nao uma solucao imaginada pela equipe.

O que implementar

- Mapear stakeholder, objetivo, poder de decisao e dor em uma matriz simples.
- Escrever a jornada: consultar -> solicitar -> aprovar -> retirar -> devolver -> manter.
- Definir RF, RNF e criterios de aceitacao para as cinco regras.
- Classificar riscos: conflito de reserva, ativo perdido, indisponibilidade e dado pessoal.

Evidencias que voce entrega

- Documento de visao de uma pagina.
- Mapa de stakeholders e jornada em fluxo.
- Backlog com 12-16 historias pequenas.
- ADR-001: monolito modular como ponto de partida.

Itens do roadmap

Responsabilidades do arquiteto, requisitos, documentacao, UML basico, BABOK e Agile/Kanban.

Revisao de arquiteto

Explique por que RB-01 e RB-04 sao as regras que merecem maior protecao tecnica.

Cuidado intencional

Nao projete microservicos. Nesta fase, o valor esta em reduzir ambiguidade.

Primeiro corte vertical: o produto existe

Por que agora?

O laboratorio aceita testar uma versao em ambiente interno. Agora voce precisa converter a jornada em software de ponta a ponta, sem tentar resolver escala prematuramente.

O que implementar

- Criar repositório, convenções, Docker Compose e dados de exemplo.
- Implementar catálogo de ativos, solicitação de reserva, fila de aprovação e devolução simples.
- Construir portal do solicitante e painel do gestor.
- Expor REST documentado para os casos de uso principais.

Evidencias que voce entrega

- Aplicação rodando localmente com um cenário demonstrável.
- OpenAPI inicial e collection de testes.
- C4: contexto e containers.
- Pipeline com lint, testes e build.

Itens do roadmap

Client/server, HTTP/HTTPS, REST, SPA/SSR inicial, containers, Git e CI/CD.

Revisão de arquiteto

Demonstre a jornada inteira para uma câmera e mostre onde cada estado muda.

Cuidado intencional

MVP não é apenas CRUD: a transição de estado precisa estar no backend, não em uma tela.

Dominio protegido por arquitetura em camadas

Por que agora?

O primeiro piloto trouxe mudancas frequentes na tela. Para que a interface nao dite as regras, voce separa o dominio das adaptacoes tecnicas.

O que implementar

- Separar dominio, aplicacao, interfaces e infraestrutura.
- Modelar Asset e Loan; codificar transicoes validas e invariantes.
- Aplicar SOLID somente onde reduz dependencia real.
- Adicionar testes unitarios e de integracao para RB-02 e RB-03.

Evidencias que voce entrega

- Diagrama de componentes e modelo de dominio.
- ADR-002: arquitetura em camadas/hexagonal leve.
- Suite de testes das regras e testes de API.
- Relatorio curto sobre acoplamento removido.

Itens do roadmap

OOP, SOLID, TDD, DDD, arquitetura em camadas, MVC/MVVM e UML de classes.

Revisao de arquiteto

Mostre que a mesma regra roda por HTTP e por teste sem subir o frontend.

Cuidado intencional

Evite DDD cerimonial: agregados e value objects so entram quando deixam uma regra mais clara.

Corretude sob concorrência: o item não duplica

Por que agora?

No primeiro dia de uso, dois solicitantes tentam reservar o mesmo projetor quase no mesmo instante. Esse é o incidente que justifica estudar transação, isolamento e idempotência.

O que implementar

- Criar restrição de empréstimo ativo por ativo e transação para aprovar/retirar.
- Definir estratégia de lock (otimista ou pessimista) e idempotency key para comandos.
- Criar teste com requisitos concorrentes e um teste de carga simples.
- Documentar como o sistema responde a repetição de um mesmo comando.

Evidências que você entrega

- ADR-003: consistência e lock.
- Teste automatizado que prova RB-01.
- Resultado de carga com taxa de erro e p95.
- Runbook para conflito de reserva.

Itens do roadmap

SQL, ACID, CAP como limite teórico, consistência eventual, performance e qualidade de dados.

Revisão de arquiteto

Defenda a escolha do isolamento e mostre que apenas uma retirada vence.

Cuidado intencional

CAP não é um botão de escolha. Use o conceito para explicar o limite, não para justificar modismo.

Seguranca, privacidade e trilha de auditoria

Por que agora?

O gestor percebe que a lista de solicitantes contem dados pessoais e que uma aprovacao sem autor identificavel nao serve para investigar um problema. Seguranca vira requisito de produto.

O que implementar

- Implementar autenticacao e RBAC para solicitante, gestor, tecnico e admin.
- Proteger senhas com hashing; aplicar sessao/token e expiracao.
- Registrar evento de auditoria para retirada, devolucao e manutencao.
- Fazer checklist OWASP: validacao, controle de acesso, rate limit e segredos.

Evidencias que voce entrega

- Modelo de ameacas leve.
- ADR-004: identidade e autorizacao.
- Tela/consulta de auditoria.
- Casos de teste de acesso negado e log de alteracao.

Itens do roadmap

Hashing, PKI, autenticacao, autorizacao, OWASP, privacidade e auditabilidade.

Revisao de arquiteto

Mostre que um solicitante nao aprova sua propria reserva e que o historico nao e editado pela UI.

Cuidado intencional

Log nao pode vaziar token, senha, documento ou detalhes desnecessarios de uma pessoa.

Contratos e integrações: REST no produto, outros protocolos no laboratório

Por que agora?

A Orion quer que um painel externo consulte disponibilidade. Um fornecedor antigo também oferece um calendário em SOAP. A necessidade real é interoperar sem contaminar o núcleo.

O que implementar

- Evoluir OpenAPI, versão de contrato e erros padronizados no REST.
- Criar consulta GraphQL somente para dashboard agregado.
- Criar pequeno serviço gRPC de disponibilidade e medir latência local.
- Implementar adapter SOAP simulado para o calendário legado; desenhar a fronteira anti-corruption.

Evidências que você entrega

- Matriz REST vs GraphQL vs gRPC vs SOAP.
- ADRs 005-007 para contratos e adaptadores.
- Testes de contrato e coleção de exemplos.
- Diagrama de sequência da integração com calendário.

Itens do roadmap

REST, GraphQL, gRPC, SOAP, ESB/BPM como conceitos de integração, contratos e API gateway.

Revisão de arquiteto

Explique por que REST continua o contrato principal apesar de você ter experimentado outros protocolos.

Cuidado intencional

GraphQL, gRPC e SOAP são laboratórios delimitados; não crie quatro interfaces de produção para a mesma operação.

Mensageria e processamento assincrono

Por que agora?

O e-mail de confirmacao deixa a aprovacao lenta e falha quando o provedor cai. A aprovacao precisa continuar correta mesmo que a notificacao aconteca depois.

O que implementar

- Publicar evento LoanApproved/LoanOverdue via outbox.
- Criar worker com retry, DLQ e idempotencia.
- Simular indisponibilidade do provedor de e-mail.
- Exibir status da notificacao apenas para operador.

Evidencias que voce entrega

- ADR-008: outbox e entrega ao menos uma vez.
- Dashboard simples de fila/DLQ.
- Teste de falha e reprocessamento.
- Runbook de mensagem parada.

Itens do roadmap

Filas, pub/sub, event-driven architecture, event sourcing como contraste, processos de negocio e confiabilidade.

Revisao de arquiteto

Desligue o e-mail e prove que aprovar uma reserva ainda funciona; depois recupere a notificacao.

Cuidado intencional

Nao transforme eventos em fonte de verdade do dominio ainda. PostgreSQL continua sendo a fonte transacional.

Tarefas agendadas, cache e serverless com justificativa

Por que agora?

O produto precisa marcar atrasos de madrugada e a consulta de catalogo ficou popular. Sao dois problemas diferentes: trabalho em lote e leitura repetida.

O que implementar

- Criar job agendado idempotente para marcar OVERDUE.
- Adicionar Redis para consultas de catalogo e invalidacao em mudanca de ativo.
- Executar o job como funcao serverless ou cron containerizado e comparar operacao/custo.
- Definir SLO inicial de leitura e monitorar cache hit rate.

Evidencias que voce entrega

- ADR-009: cache e invalidacao.
- ADR-010: runtime de tarefa agendada.
- Teste de tempo controlado para RB-05.
- Painel com p95 de consulta e taxa de acerto do cache.

Itens do roadmap

Serverless, cloud patterns, cache, disponibilidade, jobs, observabilidade e custo.

Revisao de arquiteto

Mostre o mesmo job em duas alternativas e escolha uma com base em frequencia, operacao e custo.

Cuidado intencional

Nunca use cache como controle de disponibilidade. A decisao de emprestar continua transacional.

Dados, ETL e analise operacional

Por que agora?

A diretora pede resposta para duas perguntas: quais categorias ficam mais tempo atrasadas e quais itens entram mais em manutencao? O banco transacional nao deve virar um painel pesado.

O que implementar

- Definir eventos analiticos e uma tabela de fatos pequena.
- Criar ETL incremental de PostgreSQL para area analitica.
- Publicar dashboard de utilizacao, atraso e manutencao.
- Rodar laboratorio PySpark/MapReduce com um volume simulado de eventos e comparar com SQL.

Evidencias que voce entrega

- Diagrama do fluxo de dados.
- Dicionario de metricas e linhar de dados.
- Job ETL reproduzivel.
- Nota tecnica: quando Spark faz e quando nao faz sentido.

Itens do roadmap

SQL/NoSQL, ETL, data warehouse, analytics, Hadoop, Spark e MapReduce.

Revisao de arquiteto

Reproduza um numero do dashboard a partir de uma consulta ou job rastreavel.

Cuidado intencional

Hadoop/Spark e MapReduce sao laboratorio de volume simulado; nao sao dependencia do MVP.

Limites de distribuicao: CQRS, servico e consistencia eventual

Por que agora?

O painel gerencial passou a fazer consultas agregadas caras enquanto o fluxo de retirada exige resposta imediata. Agora ha um motivo concreto para separar leitura de comando e testar um servico extraido.

O que implementar

- Criar read model de disponibilidade alimentado por eventos.
- Comparar monolito modular, CQRS interno e notificacao como servico separado.
- Usar actor/modelo de fila para serializar atualizacao de um ativo em laboratorio.
- Documentar disponibilidade, atraso de propagacao e estrategia de reconciliacao.

Evidencias que voce entrega

- ADR-011: CQRS interno e limite de servico.
- Diagrama C4 de containers atualizado.
- Teste que mostra leitura eventualmente consistente.
- Metrica de lag do read model.

Itens do roadmap

Sistemas distribuidos, CQRS, consistencia eventual, modelo de atores, SOA, microservicos e service mesh como proxima fronteira.

Revisao de arquiteto

Explique para o gestor por que a tela de relatorio pode atrasar segundos, mas a retirada nao pode emprestar em duplicidade.

Cuidado intencional

Extrair um servico e opcional para producao. O aprendizado obrigatorio e medir o custo da distribuicao.

Experiencia web: escolha de renderizacao e fronteiras de frontend

Por que agora?

Solicitantes usam celular para consultar itens; gestores usam desktop para operar filas. A interface precisa ser rapida, acessivel e coerente, sem dividir o frontend antes de haver uma necessidade.

O que implementar

- Usar SSR para paginas autenticadas que precisam de dados atuais; SSG para politicas e ajuda.
- Criar SPA parcial para calendario/filtros com estados de erro e carregamento.
- Validar acessibilidade e semantica W3C/WHATWG.
- Fazer POC de microfrontend para um painel isolado e registrar por que nao adotou (ou adotou) agora.

Evidencias que voce entrega

- ADR-012: SSR/SPA/SSG.
- Auditoria basica de acessibilidade.
- Teste E2E do fluxo de reserva.
- POC isolada de microfrontend com conclusao escrita.

Itens do roadmap

React, functional programming em UI, SPA, SSR, SSG, microfrontends, W3C/WHATWG e reactive programming.

Revisao de arquiteto

Defenda a estrategia de renderizacao por tela, nao por preferencia de framework.

Cuidado intencional

Microfrontend e experimento controlado. Uma unica aplicacao continua mais simples para este dominio.

Plataforma operavel: cloud, rede, IaC e recuperacao

Por que agora?

O piloto sai do notebook. Se o deploy falhar, se o banco cair ou se uma versao ruim subir, o laboratorio precisa saber como voltar. Arquitetura inclui operacao.

O que implementar

- Containerizar app e worker; provisionar ambiente com Terraform.
- Criar CI/CD com migration segura, health check e rollback.
- Configurar HTTPS, proxy reverso, firewall/rate limit e variaveis por ambiente.
- Instrumentar logs, metricas e traces; escrever backup e plano de recuperacao.

Evidencias que voce entrega

- Diagrama de rede TCP/IP/HTTP.
- Infraestrutura reproduzivel por IaC.
- Pipeline verde com deploy e rollback demonstrado.
- Runbook com RTO/RPO, alerta e resposta a incidente.

Itens do roadmap

Linux, Docker, cloud, IaC, CI/CD, OSI/TCP/IP, HTTP/HTTPS, proxy, firewall, observabilidade, service mesh e ITIL.

Revisao de arquiteto

Execute um rollback seguro e mostre tempo de recuperacao, p95 e taxa de erro antes/depois.

Cuidado intencional

Service mesh pode ficar em laboratorio local. Para poucos servicos, observabilidade e rede simples costumam ser mais valiosas.

Visao corporativa, integracao e defesa final

Por que agora?

A Orion considera expandir o LabTrack para outros campi. A decisao nao e apenas tecnica: dados de pessoas, ativos e manutencao podem conversar com sistemas corporativos.

O que implementar

- Desenhar contexto corporativo: directorio de usuarios, inventario/ERP e suporte/CRM simulados.
- Definir ownership de dados e contratos de integracao.
- Usar uma lente TOGAF/IAF leve: negocio, dados, aplicacoes e tecnologia.
- Preparar demo final, retrospectiva e backlog de proximos passos.

Evidencias que voce entrega

- Mapa de capacidades e contexto corporativo.
- ADR-013: ownership e integracao corporativa.
- Portfolio com 13 ADRs, diagramas, testes e runbooks.
- Apresentacao de 10 minutos e decisao de proximo investimento.

Itens do roadmap

Enterprise software (ERP/CRM), TOGAF/IAF, PMI/ITIL, governanca, comunicacao e mentoria tecnica.

Revisao de arquiteto

Apresente o LabTrack para uma diretora: problema, evidencia, trade-offs, riscos e o que voce faria com mais tres meses.

Cuidado intencional

Nao simule uma empresa inteira. Os sistemas corporativos sao contratos falsos para exercitar limites e ownership.

Checklist de encerramento

Use o buffer final para consolidar. O produto não precisa ter todas as tecnologias do mundo em produção; precisa demonstrar que você sabe escolher, isolar, medir e operar uma decisão.

Eixo	Perguntas finais
Produto	As cinco regras simples continuam verdadeiras nos fluxos normais e nos fluxos de falha?
Arquitetura	Cada decisão importante tem ADR, contexto, trade-off e plano de reversão?
Qualidade	Ha testes de regra, contrato, integração, E2E e uma carga concorrente reproduzível?
Operação	Você sabe o p95, como alertar, como fazer rollback e quanto tempo leva para recuperar?
Comunicação	Você consegue explicar em cinco minutos por que REST, fila, cache, CQRS ou serverless entram ou ficam fora?

Ritual semanal sugerido

Bloco	Tempo	Atividade
Estudo	1h30	Ler somente os conceitos do próximo problema e escrever perguntas.
Design	1h	C4/UML, ADR e definição do experimento.
Construção	3h30	Implementar um corte vertical e cobrir a regra/contrato.
Validação	1h	Executar demo, medir e registrar o que deu errado.
Defesa	30min	Explicar a decisão em voz alta ou gravar um resumo curto.

Referência de escopo

Roadmap Software Architect - roadmap.sh/software-architect. Os assuntos foram convertidos em problemas do LabTrack; quando um tópico não é natural ao produto, ele aparece como laboratório com fronteira e justificativa.

CHEGADA

O objetivo não é decorar arquitetura.

E construir critério para decidir: o que entra, o que fica de fora, como provar que funciona e como sustentar a decisão quando o sistema cresce.

Proximo passo

Comece pelo Capítulo 01. Não abra o editor antes de ter a jornada, as cinco regras e o primeiro ADR. A primeira entrega é uma decisão defendível, não uma tela.